

프론트엔드에서 풀스택, AI까지 확장해 온 Product Engineer



정다훈 Full-stack Product Engineer · 총 5년 9개월

휴대폰	010-4346-0429	Email	dahun428@naver.com
주소	경기 구리시 인창동	병역	군필 (육군 병장)
Portfolio	github.com/dahun428-fx		

요약

6년차 풀스택 개발자로, **React·Next.js** 프론트엔드부터 **Node.js·Spring Boot, MySQL, AWS**까지 전 계층에서 아키텍처를 설계하고 기술 의사결정을 주도합니다. 레거시 전환으로 커리어를 시작해 모던 웹과 풀스택으로 영역을 넓혔고, 프론트 → 컨텍스트 → 하네스 엔지니어링으로 AI 활용을 단계적으로 심화하며 팀의 AI Transformation을 이끌고 있습니다.

글로벌 B2B 커머스의 대규모 레거시 전환을 기술 리더로서 완수했고, 삼성물산에서 현장 작업자들이 쓰는 모바일 애플리케이션과 대규모 데이터 대시보드를 개발했습니다. 대웅제약에서는 AI 건강검진 챗봇을 개발해 **임직원 3,500여 명이 사용하는 서비스로 런칭**하고, 신규 고객사가 **2주 만에 도입할 수 있는 SaaS 구조**로 제품화했습니다.

핵심 성과

AI 건강검진 챗봇 제품화 — 임직원 3,500여 명 실사용, 타사 도입 가능한 SaaS 구조

- 분석·설계부터 구현·런칭까지 주도해 **3,500여 명 실사용 서비스** 완성, 신규 고객사가 **2주 만에 도입하는 SaaS 구조**로 제품화
- AI 생성 코드의 검증 병목을 직접 진단해 Vitest 약 2,600케이스·Playwright E2E 검증 하네스 단독 구축, 컨텍스트 관리·금지 패턴·보안 검증 절차를 FE AX 표준(SOP)으로 정리
- 팀원 10명 전원의 개발 표준으로 정착** — 반복 QA **3시간 → 30분**, 반복 컴포넌트 개발 **90분 → 15분**

대규모 레거시 전환 리드 — 글로벌 커머스 전면 전환과 외주 플랫폼 풀스택 내재화

- 글로벌 B2B 커머스 전면 전환(PHP·jQuery → React·Next.js·TypeScript)을 기술 리더로 완수 — 일본 본사 주도 전 지사 아키텍처 통일에서 한국 웹 전환 리드
- SSR·CSR 렌더링 경계 설계, 다국어·Datadog 관측성·UI/UX 현대화까지 서비스 전반 업그레이드
- 대웅제약 B2B 건강 플랫폼을 화면(React)부터 Spring Boot·MySQL까지 End-to-End 재구축 — 점진 전환 구조 직접 설계로 **108개 페이지 9주 단독 내재화**

대용량 데이터 플랫폼의 풀스택 성능 설계 — 수만 건 화면의 병목을 양단에서 해결

- "API가 느릴 것"이라 추정하는 대신 API 응답·화면 노출 시점을 분리 측정해 병목 특정
- 프론트 렌더링 생명주기 분리·목록 가상화와 백엔드 DB 인덱싱·DTO 경량화를 양단에서 단독 적용 — 대시보드 렌더링 **2.5초 → 1초대**, 수만 건 검색 응답 **5초 → 1초**

ML·LLM 학습에서 조직의 일하는 방식 변화까지

- ChatGPT 등장 시점에 ML·LLM 스택(LangChain·HuggingFace·RAG·VectorDB)을 자기학습, AI 소프트웨어 사이드 프로젝트로 체득해 실무 VectorDB·RAG 참여·Python 응답 로직 개선으로 연결
- 외부 컨퍼런스·전문가 인사이트로 Codex·Claude Code 활용 방안 정립, AI 기반 자동화로 팀 인력 효율 향상·표준 문서화로 사내 공식 자산화
- 세미나 12회·기술 문서 48건**으로 조직에 이식 — 팀의 일하는 방식을 AI 중심으로 전환

대웅제약 AI추진팀

2025.07 ~ 재직중

풀스택 Product Engineer / AI코치·비즈36.5·바이오에이지 신규 개발 및 유지보수 / 팀 AI Transformation 리드

AI 건강검진 챗봇을 분석·설계부터 런칭까지 주도해 **3,500여 명이 쓰는 실사용 서비스로 제품화**하고, 외주에 묶여 있던 B2B 건강 플랫폼을 풀스택으로 재구축했습니다. AI 코딩 에이전트 활용을 팀 표준으로 수립하고 검증을 자동화해 팀의 개발 방식 자체를 바꿨습니다.

- AI 건강검진 챗봇 제품화 — **3,493명 실사용 전환**, Config 기반 SaaS 구조로 신규 고객사 FE 커스터마이징 **2주 납품**
- SSE 스트리밍·Markdown 렌더링 계층·오류 가드레일 설계로 예측 불가능한 LLM 출력을 안정적인 제품 경험으로 완성
- 임직원 건강검진 통합플랫폼 풀스택 재구축 — **108개 페이지 9주 단독 전환**, MySQL·Spring Boot·REST API·React End-to-End 개발, React 전용 데이터 집계 계층(Node.js) 구축
- 팀 AI Transformation 리드 — FE AX 표준(SOP)과 검증 하네스를 팀 **10명 전원의 개발 절차로 정착**, 반복 QA **3시간 → 30분**
- 수동 FTP 배포를 Jenkins CI/CD로 전환(리드타임 **10분 → 2분**), 바이오에이지 서비스 AWS(S3·EC2) 기반 운영·유지보수

내담씨앤씨 SI사업부

2020.10 ~ 2025.06

대리 / 웹 프론트엔드 개발자 / 삼성물산·한국미스미 프로젝트

글로벌 B2B 커머스, 대용량 데이터 대시보드, React Native 앱 프로젝트에서 프론트엔드 개발을 수행했습니다. jQuery·PHP 레거시의 유지보수성·페이지 성능·모바일 사용성·데이터 시각화 문제를 React·Next.js·Vue 3·React Native로 개선하며 기술 폭을 넓혔습니다.

- 한국미스미 글로벌 B2B 커머스 Next.js 전면 전환 — 전 지사 아키텍처 통일의 한국 파트를 6인 팀 리드로 완수, 평균 로딩 **약 50% 개선**·사용자 체류 시간 **32% 증가**
- 페이지 특성별 SSR·CSR 렌더링 경계 설계와 code splitting·next/image·prefetch로 페이지 접근 시간 **8초 → 2초** 단축
- i18next 영·중·일 다국어·SEO 대응, Lighthouse·GA·Adobe Analytics·Datadog 기반 성능·SEO·오류 추적 체계 운영
- 삼성물산 대용량 데이터 대시보드(Vue 3·ECharts·RealGrid) 렌더링 생명주기 분리·Lazy Rendering으로 **2.5초 → 1초대**
- React Native 기반 iOS·Android 앱 검색 성능 **5초 → 1초**(FlatList 가상화·재렌더링 제거), 플랫폼별 이슈 대응

AI 활용 개발

Codex · Claude Code · Harness Engineering · LLM 챗봇 제품화(SSE) · AI 생성 코드 검증 SOP

- AI 생성 코드의 컨텍스트 관리·금지 패턴·보안 위험·검증 절차를 **FE AX 표준(SOP)으로 단독 설계**, 팀 10명 전원의 개발 방식으로 정착
- LLM 챗봇 제품화 전 과정 수행 — SSE 스트리밍·Markdown 렌더링 계층·오류 가드레일 설계로 예측 불가능한 LLM 출력을 안정적인 화면으로 전환
- LLM 릴레이 서버·AI 오케스트레이션 구현에 백엔드·LLM 개발자와 협업, LangChain·RAG·VectorDB 구성 참여

프론트엔드

React · Next.js · Vue 3 · React Native · TypeScript · TanStack Query · Recoil · Redux · Tailwind CSS

- 페이지 특성별 SSR·CSR 렌더링 경계 설계, Config-Driven UI·Base-Theme 기반 테넌트별 SaaS 확장 구조 구현
- 레거시(jQuery·PHP·Thymeleaf) 서비스의 전환 전략 설계 — 전면 전환과 화면 단위 점진 전환을 모두 실현 완수
- 대용량 데이터 시각화(ECharts·RealGrid)와 RN 목록 가상화 등 렌더링 성능 최적화

백엔드·데이터

Node.js(Express) · Java · Spring Boot · MySQL · REST API

- React 전용 데이터 집계 BFF(Node.js)를 신규 구축 — 모놀리식 BE-FE 분리 전략의 일환으로 RESTful 구조 점진 도입
- MySQL 데이터 모델 설계부터 Spring Boot 서버 로직·REST API까지 End-to-End 단독 개발, DB 인덱싱·DTO 경량화로 응답 최적화
- Python(FastAPI) 기반 AI 서버의 프론트 구성·응답 후처리 개선 참여

품질·검증

Playwright · Vitest · Jest · Storybook · Chromatic · ESLint

- 발화 테스트·LLM 응답 검증·타입/빌드·배포 확인을 통합한 검증 하네스를 단독 구축, 배포 파이프라인 필수 게이트로 편입
- Storybook 컴포넌트 카탈로그와 Chromatic UI 테스트로 기획·개발 협업·회귀 검증 표준화

배포·운영·관측

Jenkins · GitLab CI/CD · Vercel · AWS S3 · EC2 · Datadog · GA4 · Adobe Analytics · PostHog · Web Vitals

- 수동 FTP 배포를 Jenkins CI/CD 파이프라인으로 전환하고 배포 전후 검증 기준 수립, AWS(S3·EC2) 기반 서비스 운영
- Lighthouse·Datadog·GA·Adobe Analytics로 성능·SEO·오류를 지표화, GA4·PostHog 운영 대시보드로 비개발 직군 직접 확인 체계 구축

1. 대응제약 | AI 건강검진 챗봇 제품화

2025.07 ~ 2025.10

[주요 업무]	AI 챗봇 신규 개발에서 프론트엔드 총괄 — 아키텍처 분석·설계부터 구현·테스트까지 수행, 응답 정책·AI 서버 로직 등 제품 전반의 기술 결정에 관여
[문제]	AI 건강검진 챗봇의 신규 런칭이 필요했습니다. MVP 정의와 PoC 검증을 거쳐 10주 내 실서비스 전환이 목표였고, 프론트엔드 전체 구현과 함께 기획자·백엔드·LLM 개발자와의 협업이 필요했습니다.
[주요 실행]	• 아키텍처·정책 설계 — 기획자와 MVP 범위 정의, AI 개발자와 API 응답 형식·스트리밍·오류 정책 합의 주도로 제품 아키텍처 전체 확정 • AI 채팅 경험 구현 — SSE 기반 실시간 스트리밍과 중지·재시도·재생성·상답 이력·답변 피드백 흐름, Markdown → 표·차트·링크 React 컴포넌트 렌더링 계층, XSS·404/500/LLM 가이드라인과 오류 4유형 분류·복구 전략 구현 • 도메인 화면·데이터 기능 구현 — 건강검진 결과·건강 점수·만성질환·이상 결과·종합 소견 등 건강 데이터 분석 기능을 포함한 12개 도메인 모듈과 Feature 모듈 10종·주요 라우트 7종 구성, 브라우저 언어 감지 기반 i18n 다국어 환경 구축 • SaaS 확장 구조 도입 — 기능별 차등 과금 요구를 아키텍처로 번역, 완성된 코드베이스의 대규모 리팩토링을 감수하고 Base-Theme·Config-Driven UI(feature flag)로 전환 — 운영·기획자가 FE 단에서 고객사별 테마·기능 노출 직접 제어 • LLM 릴레이 서버·AI 오케스트레이션 구현 — 백엔드·LLM 개발자와 협업, Python 프롬프트 구성·응답 후처리 로직 개선 및 VectorDB·RAG 구축 협업
[성과]	• AI 서비스 구현 규모 — 12개 도메인 모듈, REST API 연동 31종·응답 모델 33종, Feature 모듈 10종·주요 라우트 7종, TypeScript·React 운영 코드 약 8만 LOC 구현·운영 • FE 완성도 확보 — AI Markdown 렌더링 표준화로 실사용 발화 400여 건 검증 전체 통과(All Pass), SSE·React Query 캐싱·상태관리 설계로 AI 발화 응답 평균 2초대·Lighthouse 91점 달성 • 품질 지속 향상 체계 — Vitest 테스트 약 2,600케이스(파일 315개)·Storybook 스토리 210개·Playwright E2E 시나리오 5종 구축으로 반복 QA 3시간 → 30분 단축, PostHog·Web Vitals 사용자 추적 기반 품질 관리로 사용성 4.18점·만족도 4.06점·완성도 4.05점 확보 • 실사용 서비스 전환 — 3,493명 규모 계획 대비 2주 조기 실사용 서비스 전환 • SaaS 제품화·신규 고객사 납품 — 테넌트별 콘텐츠·테마·기능 플래그를 제어하는 관리자 프리뷰 도구(JSON 편집·Diff·이력 관리) 개발로 Config-Driven SaaS 구조 완성, 신규 고객사(웰체크) FE 커스터마이징 2주 납품(최초 구현 10주 대비 5배)

기술: React · TypeScript · TanStack Query · Recoil · SSE · Vitest · Storybook · Playwright · PostHog · Node.js · Nginx · Python · FastAPI · LLM

2. 대응제약 | 임직원 건강검진 통합플랫폼 구축

2025.11 ~ 2026.02

[주요 업무]	2019년 레거시 검진 서비스를 React 기반 임직원 건강검진 통합플랫폼으로 현대화 — 서비스·DB 구조 분석부터 점진 전환 설계, MySQL·Spring Boot·REST API·화면 End-to-End 개발, 배포 자동화까지 풀스택 총괄
[문제]	외주 중심으로 운영되던 2019년 검진 예약 레거시를, 회사가 직접 개발·운영하며 고객사에 판매할 수 있는 임직원 건강검진 통합플랫폼으로 도약시켜야 했습니다. jQuery·Thymeleaf 화면과 Spring Boot 로직, MySQL 데이터가 강결합된 모놀리식이라 SPA로 한 번에 전환할 수 없었고, Web·Admin·App의 권한과 데이터 출력 정책도 제각각이었습니다.
[주요 실행]	• 서비스·데이터 구조 분석 — 기존 소스와 DB 스키마를 분석해 화면 요청부터 Controller·Service·Query·DB·출력까지 데이터 흐름 전체 정리 • 점진 전환 구조 설계 — 전면 재작성(일정·회귀 리스크)과 jQuery 장기 공존(유지비·DOM 충돌)을 모두 저울질한 끝에, React를 화면 단위로 공존·재사용하는 점진 전환 구조를 직접 설계하고 변경·장애 빈도 기준 우선순위로 전환 실행 • 풀스택 End-to-End 개발 — 신규 건강관리 기능의 MySQL 데이터 모델, Spring Boot 서버 로직, REST API, Web/Admin 화면까지 전 구간 직접 개발 • BFF 계층 신규 구축 — React 전용 데이터 집계·프록시 API를 Node.js(Express)로 구축, 모놀리식 BE-FE 분리 전략의 일환으로 RESTful 구조 점진 도입 • AI 기반 회귀 검증 — 82개 화면 전환의 UI 회귀 검증을 Playwright 기반 AI E2E 테스트로 자동화해, 대부분의 반복 UI 검증을 사람 대신 수행하는 검증 체계 적용 • 배포 자동화 — FileZilla 수동 이관·빌드에 의존하던 배포를 Jenkins 빌드·검증·배포 파이프라인으로 전환, WebView 호환성 검증 기준 수립
[성과]	• 통합플랫폼 도약 — 2019년 레거시를 React 기반 임직원 건강검진 통합플랫폼으로 재구축하고 UI/UX 전면 개편을 주도, 대응제약 대상 신규 판매 매출 1.0억 원 기여 • 단독 풀스택 전환 — 108개 페이지·82개 화면을 9주 내 단독 전환, 외주 없이는 못 고치던 서비스를 내부 개발·운영 구조로 전환 • 데이터-투-화면 E2E 오너십 — 신규 건강관리 기능 3건을 MySQL 모델부터 서버·API·화면까지 End-to-End 단독 개발 • SaaS형 상품 구조 — 고객사별 CI·메뉴·기능 노출 변경을 1주 내 대응 가능한 구조 확보 • 검증·배포 자동화 — 반복 UI 검증 대부분을 AI 기반 E2E로 수행, 수동 FTP 배포를 Jenkins CI/CD로 전환해 배포 리드타임 10분 → 2분(약 80%) 단축

기술: React · TypeScript · Java · Spring Boot · REST API · MySQL · Node.js · Express · jQuery · Thymeleaf · Playwright · Jenkins · WebView

3. 대응제약 | 팀 AI Transformation 리드 — 하네스 엔지니어링과 개발 표준화

2026년 상반기

[주요 업무]	팀의 AI 활용 개발 방식 전반을 설계·리드 — AI 생성 코드 검증을 위한 하네스 엔지니어링 구축, FE AX 표준(SOP) 단독 설계, 품질·운영 자동화와 팀 지식 자산화 주도
[문제]	AI 코딩 에이전트 도입은 됐지만 팀원마다 활용 방식이 달랐고, AI가 만든 코드를 누가 어떤 기준으로 신뢰할 것인지가 정의돼 있지 않았습니다. 그 위에 수동 발화 테스트·반복 UI 개발·배포 전후 확인·운영 지표 조회의 병목이 겹쳐 있었습니다.
[주요 실행]	• 하네스 엔지니어링 구축 — 발화 테스트·LLM 응답 검증·타입/빌드·배포 전후 확인을 자동 실행되는 검증 하네스로 통합하고, Playwright E2E 검증을 배포 파이프라인의 필수 게이트로 편입 — 사람이 판정하지 않아도 같은 기준으로 품질이 걸러지는 구조 • FE AX 표준(SOP) 단독 설계 — AI 생성 코드를 팀 전체가 같은 기준으로 검증·신뢰할 수 있도록 컨텍스트 관리·금지 패턴·보안 위험·검증 절차를 표준으로 설계, 사내 공식 문서로 채택 • 공통 컴포넌트 표준화 — Storybook으로 기획자와 UI를 조기 확정하고 재사용 컴포넌트를 카탈로그화해, 신규 화면 조립을 공통 컴포넌트 기반으로 전환 • 운영 확인 자동화 — GA4·PostHog 기반 운영 데이터 대시보드 구축, 배포 전후 상태와 운영 지표를 비개발 직군도 직접 확인 가능하게 전환 • 학습의 조직 이식 — 외부 컨퍼런스·전문가 인사이트를 스스로 흡수해 개발 세미나 12회로 팀에 이식하고, 검증 기준·노하우를 기술 문서 48건으로 자산화해 신규 합류자도 같은 기준으로 개발 가능하게 정비
[성과]	• 일하는 방식의 전환 — 10명 팀에서 단독 설계한 SOP·검증 하네스가 전원의 개발 방식으로 정착, E2E 필수 게이트로 담당자 부재 시에도 같은 기준의 검증·배포 유지 — 사람마다 다르던 AI 활용을 팀의 시스템으로 전환 • 인력 효율 향상 — 반복 UI 검증을 자동 판정으로 대체하고 반복 컴포넌트 개발 90분 → 15분(약 83%)으로 단축, 검증 인력을 개발에 재투입 • 협업 리드타임 단축 — Storybook 기반 UI 조기 확정으로 기획-개발 검증 리드타임 2일 → 1일, 기획자와의 커뮤니케이션 비용 절감 • 유지보수성 향상 — 공통 컴포넌트 카탈로그와 SOP·기술 문서 48건으로 코드 일관성·인수인계 기반 확보, 운영 데이터 확인 3단계 → 1단계 자동화

기술: Codex · Claude Code · Playwright · Storybook · Jest · ESLint · TypeScript · GA4 · PostHog · Jenkins · GitLab CI/CD

4. 삼성물산 | 대용량 데이터 플랫폼·모바일 서비스 고도화

2024.10 ~ 2025.04 · 내담씨앤씨 SI

[주요 업무]	현장 데이터 플랫폼의 Web 대시보드(Vue 3)와 현장 작업자용 모바일 앱(React Native)의 성능 개선·기능 고도화 — 병목 진단부터 FE 렌더링 설계, BE 인덱싱·응답 경량화까지 전 구간 단독 수행
[문제]	수만 건 규모의 장비·인력 데이터를 다루는 Web 대시보드와 현장 작업자용 모바일 앱에서, 화면을 여는 데 수 초가 걸리고 검색할 때마다 로딩이 반복돼 현장의 대기 시간과 운영 부담이 쌓이고 있었습니다.
[주요 실행]	- 구간 분리 측정 진단 — "API가 느릴 것"이라 추정하는 대신 API 응답 시점과 화면 노출 시점을 분리 측정해, 병목을 클라이언트 렌더링 구간으로 특정 - 대시보드 렌더링 설계 — Vue 3-ECharts-RealGrid 대용량 화면의 렌더링 생명주기 분리와 Lazy Rendering 적용 - 모바일 목록 최적화 — 효과가 가장 컸던 FlatList 가상화-renderItem 재렌더링 제거를 중심으로 안정적 key 설계·getItemLayout 적용, debounce·캐싱·중복 요청 취소(race condition 방지)는 보조 개입으로 병행 - 클라/서버 경계 설계 — 데이터 규모를 기준으로 클라이언트 필터링과 서버 검색의 경계를 정의, 역할별 데이터 조회·표현 구조 설계 - 백엔드 응답 경량화 — Spring REST API의 필터링·정렬 로직 설계, DB 인덱싱과 DTO 투영으로 응답 경량화 직접 수행
[성과]	- 대시보드 성능 개선 — FE 렌더링 생명주기 분리와 BE 인덱싱·DTO 투영을 양단에서 단독 적용해, 대용량 데이터 렌더링 2.5초 → 1초대 단축 - 모바일 검색 개선 — 수만 건 목록의 렌더링 병목을 제거해 검색 결과 노출 5초 → 1초 단축(약 80%), FlatList 가상화·재렌더링 제거가 주 개입

기술: Vue 3 · React Native · TypeScript · ECharts · RealGrid · Java · Spring · REST API · MyBatis · MySQL

5. 한국미스미 | 글로벌 B2B 커머스 Next.js 전면 전환

2022.06 ~ 2024.10 · 내담씨앤씨 SI · 3개 프로젝트

[주요 업무]

일본 본사 주도의 전 지사 아키텍처 통일 프로젝트에서, 한국 팀 React 전문 개발자로서 6인 팀의 마이그레이션과 기술 의사결정을 주도 — 사용자 화면 개발에서 시작해 아키텍처 · 성능 · 다국어 · 배포 · 모니터링까지 책임 범위를 확장

[문제]

PHP·jQuery 기반 글로벌 B2B 쇼핑몰은 화면 간 결합도가 높아 기능 확장과 유지보수가 어려웠고, 긴 초기 접근 시간과 단일 렌더링 방식으로 성능과 검색 노출을 함께 최적화하기 어려웠습니다. 일본 본사의 전 지사 아키텍처 통일 결정을 계기로, 한국 웹의 전면 전환을 이끌어야 하는 상황이었습니다.

[주요 실행]

- **한국 웹 전환 리드** — 한국 팀에서 React를 가장 깊이 다루는 개발자로서 6인 팀의 마이그레이션을 이끌고, 일본 본사 개발자들과 일본어로 직접 소통하는 크로스보더 협업으로 전면 전환을 완료까지 견인
- **렌더링 경계 판단** — 상품 · 검색 유입 페이지는 SEO와 초기 로드를 위해 SSR로, 로그인 후 상호작용 중심 페이지는 서버 부하 · 번들 비용을 고려해 CSR로 분리 — 페이지별 렌더링 비용을 기준으로 경계를 직접 판단
- **사용자 기능 구조화** — 상품 비교 · 멀티다운로드 등 복잡한 기능을 공통 컴포넌트와 Custom Hook으로 구조화하고, Redux·브라우저 저장소로 사용자 상태 일관성 확보
- **성능 최적화 직접 적용** — Lighthouse 기반 병목 분석 후 라우트 · 컴포넌트 단위 code splitting(next/dynamic), next/image 최적화, next/link prefetch, 번들 청크 분할, Lazy Loading·비동기 API 적용
- **글로벌 요구 대응 · 관측성 운영** — i18next 기반 영 · 중 · 일 다국어 구조와 SEO 대응, Lighthouse·GA·Adobe Analytics·Datadog으로 성능·SEO·오류 지표화, Vercel 배포 자동화

[성과]

- **전면 전환 완수** — 한국 법인 웹을 PHP·jQuery에서 Next.js·React·TypeScript로 전면 전환 완료, 전 지사 아키텍처 통일의 한국 파트 완수 — 기존 PHP 서비스 대비 평균 로딩 속도 **약 50% 개선**
- **성능 개선** — 유지보수 · 성능 개선 프로젝트에서 Lighthouse 병목 분석과 리소스 최적화로 페이지 접근 시간 **8초 → 2초** 단축
- **사용자 경험 개선** — 인터랙션 기능 고도화 이후 사용자 체류 시간 **32% 증가**

기술: Next.js · React · TypeScript · JavaScript · Redux · RxJS · i18next · PHP · jQuery · Twig · Vercel · Datadog · Google Lighthouse · GA · Adobe Analytics

학력

성공회대학교 일어일본학과 졸업

2010.03 ~ 2017.02

복수전공: 경영학과 · 학점: 3.8 / 4.5

여의도고등학교 졸업

2007.03 ~ 2010.02

해외경험

일본 워킹홀리데이 · 무인양품(無印良品) 도쿄 지사 근무

2018.09 ~ 2019.06

일본 현지 매장 실무를 비즈니스 일본어로 수행 — JLPT 1급 실전 활용

교육

웹 콘텐츠 개발을 위한 응용SW엔지니어링 (중앙에이치티에이㈜)

2020.03 ~ 2020.09

Java 기반 웹 애플리케이션 백엔드 · 프론트엔드 개발 과정 이수

자격증

정보처리기사 (한국산업인력공단)

2020.08

어학

영어 · TOEIC 825점

2024.05

일본어 · JLPT 1급

2018.08